UITS

Future will be better than thy past

An initiative of PHP Family

University of Information Technology & Sciences

Compiler Lab

Project Proposal (R-4)

Course Title : Compiler Lab

Course Code : CSE0613322

Submitted to:

Teacher's Name	Designation
Sonia Afroz	Lecturer
Tasneen Tanvir	Lecturer

Submitted by:

Student Name : Gaus Saraf Murady

Student ID : 0432410005101088

Semester : Autumn 2026

Batch : 55

Department : CSE

Date of Submission: 29.08.26

Experiment No. : 4

Experiment Name: Project Proposal for creating our own language by creating a minimum of 25 custom functions using C++ custom header files

---

Introduction:

In this Project Proposal, we were tasked with creating a "language of our own". While the technical details raise some feasibility concerns (i.e., building a full compiler from scratch), I have proceeded with the learnings of our last lab session by creating custom functions and syntax abstractions in C++. As such, the primary task of this proposal / report is documented in the section below, serving as the prototype documentation of this language.

Primary Project :

Title: "b1t" Programming Language

List of custom functions used in this language:

no.	Standard Functions	My Functions	Short Description
1	int main()	main	Shortened main function declaration using macros: `#define main int main()`
2	sum(int n1, int n2)	template <typename... Args> auto sum(Args... args)	Allowed multiple passage of variables as arguments unlike standard operations that can work with only 2 values.
3	sub(int n1, int n2)	template <typename T, typename... Args> auto sub(T first, Args... rest)	
4	mul(int n1, int n2)	template <typename... Args> auto mul(Args... args)	
5	divi(int n1, int n2)	template <typename T, typename... Args> auto divi(T first, Args... rest)	
6	Std::cin >>	in >>	Shortened Input and Output syntax using References & Stream Aliasing
7	Std::cout <<	out <<	

8	for (int var = (start); (var) < (end); (step))	#define loop(var, start, end, step)	Simplified the for-loop syntax so • `for (int i = 0; i < count; i++)` works as `loop(i, 0, count, i++)` • `for (int i = 0; i <= count; i++)` works as `loop2(i, 0, count, i++)` • `for (int i = 0; i > count; i++)` works as `loopr(i, 0, count, i++)` • `for (int i = 0; i >= count; i++)` works as `loop2r(i, 0, count, i++)`
9	for (int var = (start); (var) <= (end); (step))	#define loop2(var, start, end, step)	
10	a % b	rem(a, b)	Safe integer modulo / remainder
11	std::max(a, std::max(b, c))	mx(args...)	Variadic maximum and minimum across N arguments
12	std::min(a, std::min(b, c))	mn(args...)	
13	std::pow(base, exp)	pwr(base, exp)	Fast O (log n) integer/float exponentiation
14	std::abs(x)	abs_val(x)	Absolute value (| x |)
15	long long f = 1; for(int i = 1; i <= n; ++i) f *= i;	fact(n)	Factorial (n !) with negative protection
16	long long s = 0; for(int i = 1; i <= n; ++i) s += i;	sum_n(n)	Constant-time O (1) Gauss summation
17	(n % 2 == 0)	is_even(n)	Boolean check: is integer even or odd?
18	(n % 2 != 0)	is_odd(n)	
19	(n > 0)	is_pos(n)	Boolean check: is the number strictly positive or negative?
20	(n < 0)	is_neg(n)	
21	(a == b)	is_eq(a, b)	Generic equality comparison
22	T temp = a; a = b; b = temp;	swp(a, b)	In-place reference value swap
23	#include <iostream>, <vector>, etc.	#include common / common.h	Loads curated standard headers (fast compilation)
24	#include <unordered_map>, <sstream>, etc.	#include all / all.h	Loads extended standard library without duplicates, replacing <bits/stdc++.h>
25	ios_base::sync_with_stdio(false); cin.tie(NULL);	detach_C()	Detaches C stdio synchronization for high speed

Code:

main.cpp

```
#include "main.h"

main {
  detach_C();

  int count;
  out << "How many numbers do you want to calculate? ";
  in >> count;

  if (count <= 0) {
    out << "Invalid count! Please enter a number > 0." << endl;
    return 0;
  }

  vector<double> nums(count);
  loop(i, 0, count, i++) {
    out << "Enter number " << (i + 1) << ": ";
    in >> nums[i];
  }

  out << "\n--- Calculation Results (From Dynamic Input) ---" << endl;
  out << "JOG Hocche: " << sum(nums) << endl;
  out << "BIYOG Hocche: " << sub(nums) << endl;
  out << "GUN Hocche: " << mul(nums) << endl;
  out << "VAG Hocche: " << divi(nums) << endl;
  out << "MAX Hocche: " << mx(nums) << endl;
  out << "MIN Hocche: " << mn(nums) << endl;

  out << "\n--- Additional DSL Function Demonstrations ---" << endl;
  out << "Variadic Max mx(10, 45, 22, 99, 5): " << mx(10, 45, 22, 99, 5)
      << endl;
  out << "Variadic Min mn(10, 45, 22, 99, 5): " << mn(10, 45, 22, 99, 5)
      << endl;
  out << "Modulo rem(17, 5): " << rem(17, 5) << endl;
  out << "Power pwr(2, 5): " << pwr(2, 5) << endl;
  out << "Square sqr(6): " << sqr(6) << endl;
  out << "Cube cube(3): " << cube(3) << endl;
  out << "Absolute abs_val(-15): " << abs_val(-15) << endl;
  out << "Factorial fact(5): " << fact(5) << endl;
  out << "Sum 1 to 10 sum_n(10): " << sum_n(10) << endl;
  out << "Is 8 Even is_even(8): " << (is_even(8) ? "True" : "False") <<
endl;
  out << "Is 7 Odd is_odd(7): " << (is_odd(7) ? "True" : "False") << endl;
  out << "Is 5 Positive is_pos(5): " << (is_pos(5) ? "True" : "False") <<
endl;
  out << "Is -3 Negative is_neg(-3): " << (is_neg(-3) ? "True" : "False")
      << endl;
  out << "Is Equal is_eq(10, 10): " << (is_eq(10, 10) ? "True" : "False")
      << endl;

  int a = 10, b = 20;
  swp(a, b);
  out << "Swap swp(10, 20) -> a: " << a << ", b: " << b << endl;

  return 0;
}
```

main.h

```
#ifndef MAIN_H
#define MAIN_H
#define common "common.h"

#include common

#define all "all.h"

#define main int main()
using namespace std;

// detach_C
inline void detach_C() {
  ios_base::sync_with_stdio(false);
  cin.tie(NULL);
  cout.tie(NULL);
}

// in, out
inline auto &in = std::cin;
inline auto &out = std::cout;

// loop, loop2, loopr, loop2r
#define loop(var, start, end, step)
\
  for (int var = (start); (var) < (end); (step))
#define loop2(var, start, end, step)
\
  for (int var = (start); (var) <= (end); (step))
#define loopr(var, start, end, step)
\
  for (int var = (start); (var) > (end); (step))
#define loop2r(var, start, end, step)
\
  for (int var = (start); (var) >= (end); (step))

// sum
template <typename... Args> auto sum(Args... args) { return (args + ...); }

// sub
template <typename T, typename... Args> auto sub(T first, Args... rest) {
  return (first - ... - rest);
}

// mul
template <typename... Args> auto mul(Args... args) { return (args * ...); }

// divi
template <typename T, typename... Args> auto divi(T first, Args... rest) {
  auto res = static_cast<double>(first);
  bool has_zero = false;

  auto divide_one = [&](auto val) {
    if (val == 0) {
      has_zero = true;
    } else if (!has_zero) {
```

```
      res /= val;
    }
  };

  (divide_one(rest), ...);

  if (has_zero) {
    cout << "Division of 0 --> Invalid" << endl;
    return 0.0;
  }
  return res;
}

// mx
template <typename T, typename... Args>
constexpr auto mx(T first, Args... rest) {
  auto res = first;
  ((res = (rest > res ? rest : res)), ...);
  return res;
}

// mn
template <typename T, typename... Args>
constexpr auto mn(T first, Args... rest) {
  auto res = first;
  ((res = (rest < res ? rest : res)), ...);
  return res;
}

// rem
inline int rem(int a, int b) {
  if (b == 0) {
    cout << "Modulo of 0 --> Invalid" << endl;
    return 0;
  }
  return a % b;
}

// pwr
inline double pwr(double base, int exp) {
  double res = 1.0;
  long long p = exp;
  if (p < 0) {
    base = 1.0 / base;
    p = -p;
  }
  while (p > 0) {
    if (p & 1)
      res *= base;
    base *= base;
    p >>= 1;
  }
  return res;
}

// sqr
template <typename T> inline auto sqr(T x) { return x * x; }

// cube
template <typename T> inline auto cube(T x) { return x * x * x; }
```

```
// abs_val
template <typename T> inline auto abs_val(T x) { return (x < 0) ? -x : x; }

// fact
inline long long fact(int n) {
  if (n < 0) {
    cout << "Factorial of negative number --> Invalid" << endl;
    return 0;
  }
  long long res = 1;
  for (int i = 1; i <= n; ++i)
    res *= i;
  return res;
}

// sum_n
inline long long sum_n(int n) {
  if (n <= 0)
    return 0;
  return (1LL * n * (n + 1)) / 2;
}

// is_even
inline bool is_even(long long n) { return (n % 2 == 0); }

// is_odd
inline bool is_odd(long long n) { return (n % 2 != 0); }

// is_pos
inline bool is_pos(double n) { return n > 0; }

// is_neg
inline bool is_neg(double n) { return n < 0; }

// is_eq
template <typename T1, typename T2> inline bool is_eq(T1 a, T2 b) {
  return a == b;
}

// swp
template <typename T> inline void swp(T &a, T &b) {
  T temp = a;
  a = b;
  b = temp;
}

// sum (vector)
template <typename T> T sum(const vector<T> &v) {
  if (v.empty())
    return 0;
  T total = 0;
  for (const auto &x : v)
    total += x;
  return total;
}

// sub (vector)
template <typename T> T sub(const vector<T> &v) {
  if (v.empty())
```

```
    return 0;
  T res = v[0];
  for (size_t i = 1; i < v.size(); ++i)
    res -= v[i];
  return res;
}

// mul (vector)
template <typename T> T mul(const vector<T> &v) {
  if (v.empty())
    return 0;
  T total = 1;
  for (const auto &x : v)
    total *= x;
  return total;
}

// divi (vector)
template <typename T> double divi(const vector<T> &v) {
  if (v.empty())
    return 0.0;
  double res = static_cast<double>(v[0]);
  for (size_t i = 1; i < v.size(); ++i) {
    if (v[i] == 0) {
      cout << "Division of 0 --> Invalid" << endl;
      return 0.0;
    }
    res /= v[i];
  }
  return res;
}

// mx (vector)
template <typename T> T mx(const vector<T> &v) {
  if (v.empty())
    return 0;
  T m = v[0];
  for (const auto &x : v)
    if (x > m)
      m = x;
  return m;
}

// mn (vector)
template <typename T> T mn(const vector<T> &v) {
  if (v.empty())
    return 0;
  T m = v[0];
  for (const auto &x : v)
    if (x < m)
      m = x;
  return m;
}

#endif // MAIN_H
```

common.h

```
#ifndef COMMON_H
#define COMMON_H

// --- Commonly Used Standard C++ Headers ---
#include <iostream>   // For std::cin, std::cout, std::endl, std::cerr
#include <vector>     // For std::vector dynamic array
#include <string>     // For std::string
#include <cmath>      // For math functions: sqrt, abs, pow, sin, cos,
floor, ceil
#include <algorithm>  // For std::sort, std::reverse, std::min, std::max,
std::binary_search
#include <numeric>    // For std::accumulate, std::gcd, std::lcm, std::iota
#include <iomanip>    // For std::setprecision, std::setw, std::fixed
#include <map>        // For std::map, std::multimap
#include <set>        // For std::set, std::multiset, std::unordered_set
#include <queue>      // For std::queue, std::priority_queue
#include <stack>      // For std::stack
#include <deque>      // For std::deque
#include <utility>    // For std::pair, std::make_pair
#include <climits>    // For INT_MAX, INT_MIN, LLONG_MAX
#include <cstdint>    // For int64_t, uint64_t

#endif // COMMON_H
```

all.h

```
#ifndef ALL_H
#define ALL_H

#pragma push_macro("all")
#pragma push_macro("common")
#undef all
#undef common

//
================================================================================
=
// ALL_H: Extended Standard C++ Library Headers
// (Excludes headers already included in common.h to prevent redundancy)
//
================================================================================
=

// --- 1. Additional Containers & Data Structures ---
#include <array>           // For std::array (fixed-size compile-time
arrays)
#include <bitset>          // For std::bitset (fixed-size bit arrays & bit
manipulation)
#include <forward_list>    // For std::forward_list (singly-linked list)
#include <list>            // For std::list (doubly-linked list)
#include <unordered_map>   // For std::unordered_map,
std::unordered_multimap (hash map)
#include <unordered_set>   // For std::unordered_set,
std::unordered_multiset (hash set)

// --- 2. String Streams, Views & Text Processing ---
#include <sstream>         // For std::stringstream, std::istringstream,
std::ostringstream
#include <string_view>     // For std::string_view (C++17 zero-copy string
views)
#include <regex>           // For std::regex, std::smatch,
std::regex_search, std::regex_replace
#include <charconv>        // For std::from_chars, std::to_chars (C++17
fast number parsing)

// --- 3. File System & File I/O ---
#include <fstream>         // For std::ifstream, std::ofstream,
std::fstream (file I/O)
#include <filesystem>      // For std::filesystem::path (C++17 filesystem
operations)

// --- 4. Utilities, Tuples, Smart Pointers & Type Traits ---
#include <functional>      // For std::function, std::bind, std::greater,
std::hash
#include <memory>          // For std::unique_ptr, std::shared_ptr,
std::make_unique
#include <optional>        // For std::optional, std::nullopt (C++17
optional values)
#include <variant>         // For std::variant, std::get,
std::holds_alternative (C++17 type-safe union)
#include <any>             // For std::any, std::any_cast (C++17 type-safe
container for any type)
#include <tuple>           // For std::tuple, std::make_tuple, std::tie,
std::get
```

```
#include <type_traits>     // For std::is_same_v, std::enable_if_t,
compile-time introspection
#include <typeinfo>        // For typeid, std::type_info

// --- 5. Random Numbers, Timing, Math & Numeric Limits ---
#include <random>          // For std::mt19937,
std::uniform_int_distribution, random engines
#include <chrono>          // For std::chrono high-precision clocks,
duration, time points
#include <complex>         // For std::complex numbers
#include <valarray>        // For std::valarray (vectorized math
operations)
#include <ratio>           // For std::ratio (compile-time rational
arithmetic)
#include <limits>          // For std::numeric_limits<T>::max(), min(),
epsilon()

// --- 6. Concurrency, Multithreading & Atomics ---
#include <thread>          // For std::thread, std::this_thread::sleep_for
#include <mutex>           // For std::mutex, std::lock_guard,
std::unique_lock
#include <condition_variable> // For std::condition_variable
#include <future>          // For std::async, std::future, std::promise
#include <atomic>          // For std::atomic<T> (lock-free thread-safe
variables)

// --- 7. Error Handling & Diagnostics ---
#include <exception>       // For std::exception
#include <stdexcept>       // For std::runtime_error,
std::invalid_argument, std::out_of_range
#include <system_error>    // For std::error_code, std::system_error
#include <cassert>         // For assert() macro

// --- 8. C-Compatibility Libraries ---
#include <cstdio>          // For printf, scanf, sprintf, FILE*
#include <cstdlib>         // For malloc, free, exit, rand, srand, atoi
#include <cstring>         // For strlen, strcmp, strcpy, memset, memcpy
#include <cctype>          // For isdigit, isalpha, tolower, toupper
#include <ctime>           // For time(), clock(), difftime()
#include <cfloat>          // For FLT_MAX, DBL_MAX float limits
#include <cstddef>         // For size_t, ptrdiff_t, nullptr_t

#pragma pop_macro("common")
#pragma pop_macro("all")

#endif // ALL_H
```

Discussion:

Through this assignment, I learned how to work with custom header files, functions and definitions of built-in keywords of C++. I successfully created a modified version of C++ called “b1t” programming language that shortens some algorithm uses, and conventional practices like Python. While leaving room for future improvements to be done in the future, I have documented the learnings of this project at: Compiler-Lab-Codes/Assignments/A4_b1t_Lang at main · b1tranger/Compiler-Lab-Codes

Conclusion:

I was able to complete the assignment successfully. The newly created language is usable in regular practices, but lacks universal usability due to many unhandled edge cases that will require many iterations of testing and reviewing.